

Практика 2: Реализация спроектированного приложения

Table of contents

Практика 2: Реализация спроектированного приложения	1
Предисловие	1
Задание	1
Структура отчёта	2

Практика 2: Реализация спроектированного приложения

Предисловие

Во второй части реализуется проект из Практики 1. Главная цель — превратить архитектурную схему в работающий код и убедиться, что **соответствие между схемой и кодом** прослеживается явно: каждый блок на схеме соответствует конкретному пакету/модулю, каждая стрелка — конкретному `import/require/using`.

Подключение базы данных. Вы можете подключить PostgreSQL, SQLite или любую другую СУБД — но это **не обязательно**. Реализации в памяти (`map/slice/dict`) достаточно. Смысл практики не в настройке БД, а в том, чтобы зависимости в коде соответствовали зависимостям на схеме. Если ваша архитектура спроектирована правильно, замена репозитория в памяти на PostgreSQL потребует изменения ровно одного файла — реализации интерфейса.

На что будем смотреть в первую очередь:

- Какой компонент от чего зависит (`import`'ы, конструкторы, DI-конфигурация).
- Совпадает ли направление зависимостей в коде с направлением стрелок на схеме из Практики 1.
- Зависит ли сценарий приложения от интерфейса репозитория, а не от конкретной реализации.
- Нет ли “коротких путей” (`handler` напрямую обращается к хранилищу, минуя сценарий приложения).

Задание

1. **Реализуйте слои из вашего проекта:** `domain`, `use cases`, `adapters`, `infrastructure`.
2. **Хранение данных:** в памяти (`map/slice/dict`) или реальная СУБД — на ваш выбор. Если выбрали СУБД, убедитесь, что сценарии приложения зависят от интерфейса репозитория, а не от конкретной реализации (ORM, SQL-драйвер и т.д.).
3. **Проверка `endpoint`'ов:**

- Каждый endpoint должен быть проверен через **Insomnia** или **Postman** (не Swagger UI). В отчёт — скриншоты запросов и ответов.
 - Для endpoint'ов, которые **изменяют состояние** (создание, обновление, удаление): проверьте успешный сценарий + сценарий с логической ошибкой (например, попытка оформить заказ при пустой корзине).
 - Для endpoint'ов, которые **только читают данные** (получение списка, получение по ID): достаточно успешного сценария.
 - Сценарии “ошибка авторизации” не требуются.
4. **Интеграционные тесты (необязательно, но приветствуются):**
- Если пишете тесты — минимум один тест на полный сценарий из нескольких шагов.
 - Наличие интеграционных тестов учитывается как плюс при оценке.
5. **Подготовьте отчёт** (структура ниже).

Язык: Go, Java, C#, TypeScript или Elixir/Phoenix.

Структура проекта: можно выбрать свою. Ниже только ориентир, а не жёсткий шаблон. Важно лишь, чтобы проект не сводился к одному файлу: должно быть хотя бы несколько файлов или модулей с понятным разделением ответственности.

```

app/
├─ domain/
├─ usecases/
├─ adapters/
├─ repositories/in_memory/  # или repositories/postgres/
├─ infrastructure/
└─ main.*

```

Структура отчёта

1. **Введение.** Краткое описание продукта и сценария работы пользователя (из Практики 1). Если что-то изменилось — укажите что и почему.
2. **Схема архитектуры.** Обновлённая схема из Практики 1 (если менялась). На схеме выделены слои, компоненты, интерфейсы и направление зависимостей.
3. **Соответствие «схема → код».** Таблица, связывающая каждый блок на схеме с конкретным файлом/пакетом в проекте:

Слой	Компонент на схеме	на Файл / пакет	Зависит от (import'ы)
Domain	Order	domain/order.go	—
Use Cases	CreateOrderUC	usecases/ create_order.go	domain, ports.OrderRepo
Ports	OrderRepo (интерфейс)	ports/ order_repo.go	domain

Слой	Компонент на схеме	Файл / пакет	Зависит (import'ы) от
Adapters	OrderHandler	<code>adapters/http/order.go</code>	<code>usecases</code>
Infrastructure	InMemoryOrder-Repo	<code>repositories/memory/order.go</code>	<code>domain, ports</code>

Для каждой строки важно показать, что зависимости направлены **внутрь** (к domain), а не наружу.

4. **Таблица слоёв.** Сводная таблица с количественными характеристиками:

Слой	Ответственность	LOC
Domain	Сущности, value objects, бизнес-правила	~120
Use Cases	Сценарии, оркестрация, бизнес-логика	~200
Ports	Интерфейсы репозитория и внешних сервисов	~40
Adapters	HTTP-обработчики, маршрутизация, DTO	~150
Infrastructure	Реализации репозитория, DI-конфигурация, <code>main</code>	~100

5. **Проверка endpoint'ов.** Для каждого endpoint — скриншот из Insomnia/Postman: запрос и ответ. Для мутирующих endpoint'ов — дополнительно скриншот сценария с ошибкой. Если написаны интеграционные тесты — опишите сценарии и как их запустить.

6. **Рефлексия.** Что изменилось по сравнению с проектом из Практики 1 и почему. Примеры вопросов для рефлексии:

- Появились ли новые компоненты, которых не было на схеме?
- Пришлось ли менять API-контракты?
- Какие архитектурные решения оказались удачными, какие — нет?

Что сдаётся:

1. Работающее приложение по проекту из Практики 1.
2. Реализации всех репозитория (в памяти или СУБД).

3. Скриншоты проверки каждого endpoint'а в Insomnia/Postman.
4. Интеграционные тесты (опционально).
5. Отчёт по структуре выше.

Теория к сдаче:

Вся теория из Практики 1, плюс вопросы по собственному коду:

- Покажите на своём коде: где domain, где use cases, где adapters? Покажите направление зависимостей
- Что такое инверсия зависимостей? Покажите: сценарий приложения зависит от интерфейса, а не от конкретной реализации репозитория
- Если завтра нужно заменить хранилище в памяти на PostgreSQL — какие файлы меняются, какие нет? Почему?
- Что такое прокси-компонент? Есть ли в вашем приложении непродуктивные слои?
- Почему вы выбрали именно этот протокол? В каком случае выбрали бы другой?